Lambda Expressions

What is a lambda expression?

A lambda expression is an anonymous function — a concise way to represent a block of code that can be passed around and executed later. Lambdas exist to provide implementations of *functional interfaces* (interfaces with a single abstract method).

Syntax forms

- **No parameter:**

```
() -> System.out.println("Hello from lambda");
```

- **Single parameter (parentheses optional):**

```
x -> x * x
(n) -> n * n
```

- **Multiple parameters with body block:**

```
(a, b) -> {
    int sum = a + b;
    return sum;
}
```

- **Full form with types (usually inferred):**

```
(int a, int b) -> a + b
```

Target typing & type inference

A lambda expression has no standalone type: the compiler infers the target type from the *context* — typically the type of the variable or parameter where the lambda is assigned (a functional interface).

Example:

```
Comparator<String> cmp = (s1, s2) -> s1.length() - s2.length();
```

Here the lambda is assigned to a `Comparator<String>` so the parameter types are inferred.

Closures & "effectively final"

Lambdas can capture local variables from their enclosing scope, but only when those variables are *effectively final* — you don't need to mark them `final`, but you must not reassign them after initialization.

```
int threshold = 10; // effectively final
Predicate<Integer> p = v -> v > threshold;
```

Attempting to modify the `threshold` after the lambda definition will cause a compile error.

`this` & anonymous class differences

Inside a lambda, `this` refers to the surrounding class instance — **not** to a synthetic lambda object. This differs from anonymous inner classes where `this` refers to the anonymous class instance.

Examples

Runnable using anonymous class vs lambda:

```
// before Java 8
new Thread(new Runnable() {
    @Override
    public void run() {
        System.out.println("running...");
    }
}).start();

// with lambda
new Thread(() -> System.out.println("running..."))
    .start();
```

Predicate filter:

```
List<String> names = Arrays.asList("Asha", "Ravi", "Kiran", "Meera");
List<String> longNames = names.stream()
    .filter(s -> s.length() > 4)
    .collect(Collectors.toList());
```

When to use lambdas

- Short behavior implementations (comparators, listeners).
- Compose operations (map/filter/reduce) in Stream pipelines.
- Avoid when the logic is complex — prefer named functions for readability.

Method References

Method references are compact forms of lambdas that call an existing method. They increase readability when the lambda just calls a method.

Forms

1. **Static method reference:** `ClassName::staticMethod`
2. **Instance method reference (specific object):** `instance::instanceMethod`
3. **Instance method reference (unbound):** `ClassName::instanceMethod` — maps to a lambda where the first parameter becomes the method receiver.
4. **Constructor reference:** `ClassName::new`

Examples

```
// static method reference
Function<String, Integer> parseInt = Integer::parseInt;

// specific instance
PrintStream out = System.out;
Consumer<String> printer = out::println;

// unbound instance method (the first arg is the receiver)
BiPredicate<String, String> equals = String::equalsIgnoreCase;

// constructor reference
```

```
Function<String, Person> create = Person::new; // Person(String name)
```

Ambiguity & overloads

When using method references, watch for overloaded target methods: the compiler must be able to choose the correct signature. If ambiguous, use explicit lambdas or cast the method reference to the desired functional interface.